

Complete Guide

ML Training with Scikit-Learn

Classification, Regression & Text/NLP Pipelines

Train, compare, and deploy sklearn models step by step

With complete code and working examples

Table of Contents

1. When to Use sklearn vs LLM Fine-Tuning
2. Installation & Setup
3. Step 1: Prepare Your Data
4. Step 2: Train Classification Models
5. Step 3: Train Regression Models
6. Step 4: Train Text/NLP Models
7. Step 5: Make Predictions
8. Complete Training Script Reference
9. Model Comparison & Selection Guide
10. Tips & Best Practices

1. When to Use sklearn vs LLM Fine-Tuning

	sklearn	LLM Fine-Tuning (Ollama)
Task	Classification, regression, clustering, tabular data	Text generation, chat, summarization, reasoning
Data	Structured (CSV, tables) 100-100K rows	Unstructured text 100-10K conversations
Hardware	CPU is fine No GPU needed	GPU required 8-80 GB VRAM
Training	Seconds to minutes	Minutes to hours
Output	Predictions (labels, numbers)	Generated text
Best For	Business analytics, fraud, spam detection, pricing	Chatbots, content gen, code assistants

Tip: If your data fits in a spreadsheet and you need predictions (yes/no, price, category), use sklearn. If you need the model to generate text or hold conversations, use LLM fine-tuning.

2. Installation & Setup

Install the required packages:

```
pip install scikit-learn pandas numpy joblib xgboost
```

Verify installation:

```
import sklearn
print(sklearn.__version__) # Should be 1.3+
```

Note: sklearn runs on CPU by default. No GPU or CUDA required. Works on any machine (laptop, server, cloud).

3. Step 1: Prepare Your Data

For Classification / Regression (Tabular Data)

Create a CSV with feature columns and one target column:

```
age,salary,department,experience,promoted
28,45000,engineering,3,yes
35,72000,marketing,8,yes
22,32000,sales,1,no
45,95000,engineering,15,yes
31,55000,marketing,5,no
```

For Text Classification (NLP)

```
text,label
"This product is amazing, love it!",positive
"Terrible quality, would not buy again",negative
"Great value for the price",positive
"Worst purchase ever",negative
```

Loading Data in Code

```
import pandas as pd

# CSV
df = pd.read_csv('data.csv')

# JSON
df = pd.read_json('data.json')

# Excel
df = pd.read_excel('data.xlsx')

# Quick look at the data
print(df.shape)          # (rows, columns)
print(df.head())         # First 5 rows
print(df.describe())     # Statistics
print(df.isnull().sum()) # Missing values
```

4. Step 2: Train Classification Models

One-Command Training

```
python -m sklearn_pipeline.train \
  --task classification \
  --input data.csv \
  --target label_column \
  --output ./sklearn_output/my_model
```

This automatically trains 7 models, compares them, and saves the best one.

Models Trained Automatically

Model	Strengths	Speed
Logistic Regression	Simple, interpretable, good baseline	Very Fast
Random Forest	Handles non-linear data, robust	Fast
Gradient Boosting	High accuracy, handles complex patterns	Moderate
XGBoost	State-of-the-art for tabular data	Moderate
SVM	Works well with high-dimensional data	Slow on large data
KNN	Simple, no training phase	Slow at prediction
Decision Tree	Most interpretable, visual	Very Fast

Complete Classification Code (Step by Step)

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler, LabelEncoder
from sklearn.ensemble import RandomForestClassifier
from sklearn.metrics import accuracy_score, classification_report

# 1. Load data
df = pd.read_csv('data.csv')

# 2. Separate features and target
X = df.drop(columns=['target_column'])
y = df['target_column']

# 3. Encode string labels to numbers
le = LabelEncoder()
y = le.fit_transform(y)

# 4. Handle categorical columns
X = pd.get_dummies(X, drop_first=True)
```

```

# 5. Split into train/test
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42, stratify=y
)

# 6. Scale features
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)

# 7. Train model
model = RandomForestClassifier(n_estimators=100, random_state=42)
model.fit(X_train, y_train)

# 8. Evaluate
y_pred = model.predict(X_test)
print(f'Accuracy: {accuracy_score(y_test, y_pred):.4f}')
print(classification_report(y_test, y_pred,
    target_names=le.classes_))

# 9. Save model
import joblib
joblib.dump(model, 'model.joblib')
joblib.dump(scaler, 'scaler.joblib')
joblib.dump(le, 'label_encoder.joblib')

```

5. Step 3: Train Regression Models

One-Command Training

```
python -m sklearn_pipeline.train \
    --task regression \
    --input housing.csv \
    --target price
```

Models Trained Automatically

Model	Best For	Metric
Linear Regression	Linear relationships	R2, RMSE
Ridge	Multicollinearity	R2, RMSE
Random Forest	Non-linear patterns	R2, RMSE
Gradient Boosting	Complex relationships	R2, RMSE
XGBoost	Tabular data competitions	R2, RMSE
SVR	Small datasets, non-linear	R2, RMSE
Decision Tree	Interpretability needed	R2, RMSE

Complete Regression Code

```
import pandas as pd
from sklearn.model_selection import train_test_split
from sklearn.preprocessing import StandardScaler
from sklearn.ensemble import GradientBoostingRegressor
from sklearn.metrics import r2_score, mean_absolute_error
import numpy as np

# 1. Load and split
df = pd.read_csv('housing.csv')
X = df.drop(columns=['price'])
y = df['price']
X = pd.get_dummies(X, drop_first=True)
X_train, X_test, y_train, y_test = train_test_split(
    X, y, test_size=0.2, random_state=42
)

# 2. Scale and train
scaler = StandardScaler()
X_train = scaler.fit_transform(X_train)
X_test = scaler.transform(X_test)
model = GradientBoostingRegressor(
    n_estimators=100, random_state=42
```



```
)  
model.fit(X_train, y_train)  
  
# 3. Evaluate  
y_pred = model.predict(X_test)  
print(f'R2:    {r2_score(y_test, y_pred):.4f}')
```

print(f'MAE: {mean_absolute_error(y_test, y_pred):.4f}')

```
rmse = np.sqrt(np.mean((y_test - y_pred)**2))  
print(f'RMSE: {rmse:.4f}')
```

6. Step 4: Train Text/NLP Models

One-Command Training

```
python -m sklearn_pipeline.train \
    --task text \
    --input reviews.csv \
    --text-col review \
    --target sentiment
```

How It Works: TF-IDF + Classifier

Text is converted to numbers using TF-IDF (Term Frequency-Inverse Document Frequency), which scores how important each word is to a document relative to the entire corpus. Then a classifier is trained on these features.

Complete Text Classification Code

```
import pandas as pd
from sklearn.feature_extraction.text import TfidfVectorizer
from sklearn.linear_model import LogisticRegression
from sklearn.pipeline import Pipeline
from sklearn.model_selection import train_test_split
from sklearn.metrics import classification_report

# 1. Load data
df = pd.read_csv('reviews.csv')
X_train, X_test, y_train, y_test = train_test_split(
    df['review'], df['sentiment'],
    test_size=0.2, random_state=42
)

# 2. Build pipeline (TF-IDF + model in one object)
pipeline = Pipeline([
    ('tfidf', TfidfVectorizer(
        max_features=10000,
        ngram_range=(1, 2),    # unigrams + bigrams
        min_df=2,
        max_df=0.95,
        sublinear_tf=True,
    )),
    ('model', LogisticRegression(
        max_iter=1000, random_state=42
    )),
])

# 3. Train
pipeline.fit(X_train, y_train)

# 4. Evaluate
y_pred = pipeline.predict(X_test)
print(classification_report(y_test, y_pred))
```

```
# 5. Predict new text
print(pipeline.predict(['This product is amazing!']))
# Output: ['positive']

# 6. Save the entire pipeline
import joblib
joblib.dump(pipeline, 'text_model.joblib')
```

Tip: The Pipeline object bundles TF-IDF + model together, so you only need to save/load one file. Pass raw text directly - no manual preprocessing needed.

7. Step 5: Make Predictions

Single Prediction (Classification)

```
python -m sklearn_pipeline.predict \
    --model-dir ./sklearn_output/iris \
    --input '{"sepal length (cm)": 5.1, "sepal width (cm)": 3.5,
            "petal length (cm)": 1.4, "petal width (cm)": 0.2}'

# Output:
# {
#   "prediction": "setosa",
#   "probabilities": {"setosa": 1.0, "versicolor": 0.0, ...}
# }
```

Single Prediction (Text)

```
python -m sklearn_pipeline.predict \
    --model-dir ./sklearn_output/sentiment \
    --input "This product is great!"

# Output:
# {
#   "prediction": "positive",
#   "probabilities": {"negative": 0.42, "positive": 0.58}
# }
```

Batch Prediction from File

```
python -m sklearn_pipeline.predict \
    --model-dir ./sklearn_output/iris \
    --input new_data.csv \
    --output predictions.csv
```

Loading a Saved Model in Your Own Code

```
import joblib

# Load
model = joblib.load('sklearn_output/iris/model.joblib')
scaler = joblib.load('sklearn_output/iris/scaler.joblib')
le = joblib.load('sklearn_output/iris/label_encoder.joblib')

# Predict
import numpy as np
X_new = np.array([[5.1, 3.5, 1.4, 0.2]])
X_scaled = scaler.transform(X_new)
pred = model.predict(X_scaled)
label = le.inverse_transform(pred)
print(label) # ['setosa']
```

8. Complete Training Script Reference

All CLI Options

Flag	Required	Default	Description
--task	Yes	-	classification, regression, or text
--input	Yes	-	Path to CSV, JSON, JSONL, XLSX, or Parquet
--target	Yes	-	Target column name
--text-col	text only	-	Text column name (required for task=text)
--test-size	No	0.2	Fraction of data for testing (0.1 to 0.4)
--output	No	./sklearn_output	Where to save the trained model

Output Files

File	Description
model.joblib	Trained model (or Pipeline for text tasks)
scaler.joblib	StandardScaler (classification/regression only)
label_encoder.joblib	LabelEncoder (when target is categorical)
metadata.json	Model name, metrics, feature names, config

9. Model Comparison & Selection Guide

Choosing the Right Model

Scenario	Recommended Model	Why
< 1K rows	Logistic Regression / Linear Regression	Simple models generalize better on small data
1K-100K rows	Gradient Boosting / XGBoost	Best accuracy on medium datasets
> 100K rows	SGD / Random Forest	Scales well to large datasets
Need interpretability	Decision Tree / Logistic Regression	Easy to explain to stakeholders
Text classification	Logistic Regression + TF-IDF	Fast and effective for most NLP tasks
Many features	Random Forest / XGBoost	Handles high-dimensional data well

Evaluation Metrics Explained

Metric	Task	Meaning	Good Value
Accuracy	Classification	% of correct predictions	> 0.85
F1 Score	Classification	Balance of precision & recall	> 0.80
R2	Regression	% of variance explained	> 0.70
MAE	Regression	Average absolute error	As low as possible
RMSE	Regression	Root mean squared error	As low as possible

10. Tips & Best Practices

Data Preparation

- **Handle missing values** - fill with median (numbers) or mode (categories)
- **Remove duplicates** - `df.drop_duplicates()`
- **Check class balance** - imbalanced classes hurt accuracy
- **Feature engineering** - create new features from existing ones

```
# Handle imbalanced classes
from sklearn.utils import class_weight
weights = class_weight.compute_class_weight(
    'balanced', classes=np.unique(y_train), y=y_train
)
model = RandomForestClassifier(
    class_weight='balanced', random_state=42
)
```

Improve Model Performance

- **Cross-validation** - use 5-fold CV instead of single train/test
- **Hyperparameter tuning** - use `GridSearchCV` or `RandomizedSearchCV`
- **Feature selection** - remove irrelevant features

```
# Hyperparameter tuning example
from sklearn.model_selection import GridSearchCV

param_grid = {
    'n_estimators': [100, 200, 500],
    'max_depth': [3, 5, 10, None],
    'min_samples_split': [2, 5, 10],
}

grid = GridSearchCV(
    RandomForestClassifier(random_state=42),
    param_grid, cv=5, scoring='f1_weighted', n_jobs=-1
)
grid.fit(X_train, y_train)
print(f'Best params: {grid.best_params_}')
print(f'Best score: {grid.best_score_: .4f}')
best_model = grid.best_estimator_
```

sklearn vs LLM: Decision Flowchart

- Is your data in a table/spreadsheet? -> **sklearn**
- Do you need to generate text? -> **LLM fine-tuning**
- Is your task classification/regression? -> **sklearn**
- Do you need conversational AI? -> **LLM fine-tuning**

- Budget under \$0 for compute? -> **sklearn** (runs on CPU)

All code is available in **ollama_fine_tune/sklearn_pipeline/**. Run the demo with: **python -m sklearn_pipeline.demo**